KU LEUVEN

Geavanceerde Computerarchitectuur
4th Session

Optimization techniques,
Coalesced memory access and device (GPU) memory types

Thomas Feys, thomas.feys@kueluven.be
DRAMCO – WaveCore – ESAT
Academic year 2024/25

0 Outline

1. Intermezzo
2. GPU memory access and memory types
3. Exercise
4. Appendix
5. Sources

1 Outline

1. Intermezzo
2. GPU memory access and memory types
3. Exercise
4. Appendix
5. Sources

1 Free allocated memory

```
#define SIZE 16
...
int* p1 = (int*)malloc(SIZE*sizeof(int));
int* p2;
cudaMalloc(&p2, SIZE*sizeof(int));
...
free(p1);
cudaFree(p2);
```

Otherwise memory leaks are introduced.

Can use Valgrind to detect leaks.

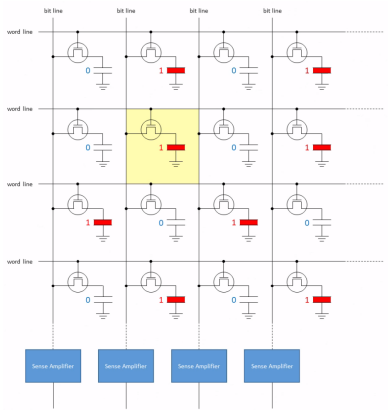
2 Outline

1. Intermezzo
2. GPU memory access and memory types
3. Exercise
4. Appendix
5. Sources

2 Coalesced memory access

Coalesce: to come together to form one larger group, substance, etc.

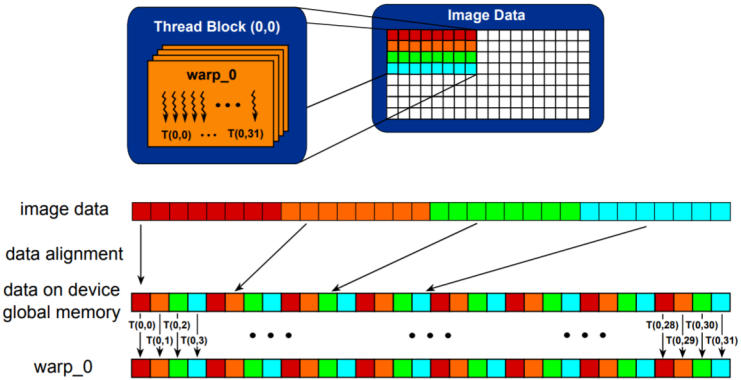
- DRAM access is done in groups, typically 32, 64, or 128 bytes (high-throughput parallel access)
- DRAM access is otherwise sequential (e.g. per request)
- Group memory access reduces memory latency



2 Coalesced memory access

- Is achieved when all threads in a warp access consecutive global memory locations.
- Threads in the same warp access global memory in a way that the hardware can combine (coalesce) their requests into as few memory transactions as possible.
- For example, for a given load instruction of a warp, if thread 0 accesses global memory location N, thread 1 location N+1, thread 2 location N+2, and so on, all these accesses will be coalesced into a as few memory requests as possible.
- Such coalesced access allows the DRAMs to deliver data as a burst.

2 Coalesced memory access in practice



2 Non-divergent threads and coalesced memory access

Non-divergent threads

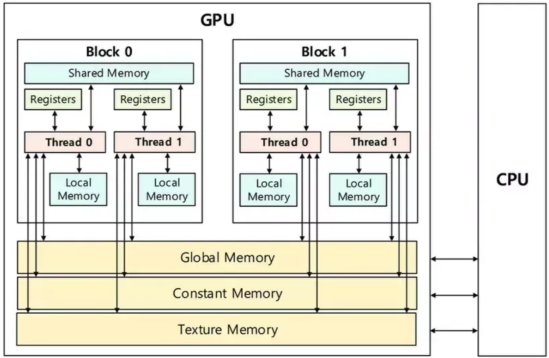
Threads in a **warp** take the same **computation** path (same complexity)

Coalesced memory access

Threads in a **warp** access the same **memory** group (adjacent addresses)

2 GPU memory types

- Shared – Per block, on-chip
- Registers – Per thread, on-chip
- Local – Per thread, off-chip
- Constant – For all, off-chip
- Global – For all, off-chip
- Texture – For all, off-chip



2 Global memory

- Off-device – high access latency
- All threads in the grid have access, as well as the CPU
- Large size – typically several GB
- The 'default' option

As in previous labs

```
1 int* garr;
2 cudaMalloc(&garr, SIZE*sizeof(int));
3 // or (see analogy to constant memory in a later slide)
4 __device__ int garr[SIZE];
```

2 Shared memory

- On-device – low access latency
- Shared among the threads of a single block
- User-managed cache, by default 48 kB (`cudaFuncSetCacheConfig(...)`)
- Two options in CUDA, see below

Hardcode `SIZE` (or use `#define`)

`my_kernel<<<blk, tpb>>>(...);` → `__shared__ int smem[SIZE];`

Define size dynamically during kernel invocation

`my_kernel<<<blk, tpb, size>>>(...);` → `extern __shared__ int smem[];`

2 Workflow with shared memory

Threads migrate data from global to shared memory

1. (on CPU) Copy data **to global memory** and run the kernel
2. (on GPU, per thread) Migrate one or more elements **to shared memory**
3. **Process** the data in **shared memory**
4. Migrate the data **back to global memory**
5. (on CPU) Copy data from global memory

2 Workflow with shared memory

Threads migrate data from global to shared memory

1. (on CPU) Copy data **to global memory** and run the kernel
2. (on GPU, per thread) Migrate one or more elements **to shared memory**
3. **Process** the data in **shared memory**
4. Migrate the data **back to global memory**
5. (on CPU) Copy data from global memory

Where should we put `__syncthreads()`?

2 Constant memory

- Off-device, optimized for low-latency read
- All threads in the grid have access, as well as the CPU
- Small size – 64 KB

From the documentation:

```
__constant__ int cmem[SIZE]; // GPU
...
int* arr; // CPU
...
cudaMemcpyToSymbol(cmem, arr, SIZE * sizeof(int)); // From CPU to GPU
```

2 Other memory

Registers

- On-device – Low access latency
- Access per thread
- `threadIdx.x`

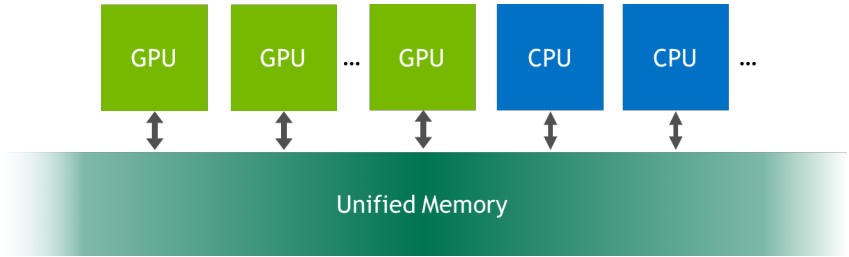
Local memory

- Off-device – High(er) access latency
- Access per thread
- `int arr[128];`

Texture memory

- Off-device with optimized read access
- Access for all threads
- For 2D graphic data

2 Regarding a question



Unified memory allows one to allocate once and use on both devices. (Abstraction that handles memory migration.)

More in the docs and a blog post.

3 Outline

1. Intermezzo
2. GPU memory access and memory types
3. Exercise
4. Appendix
5. Sources

3 Assignment

Part 1 – Coalesced memory access

- Design the following two kernels that operate on a square matrix $\mathbf{A}$ of size $N \times N$.
- Copy Kernel: $\mathbf{B} = \mathbf{A}$ (copy the input matrix to another matrix)
- Transpose Kernel: $\mathbf{B} = \mathbf{A}^T$ (transpose the input matrix)
- The matrix should be stored in **row-major order** as a contiguous 1D array (e.g., using `std::vector<float>`).
- Element $\mathbf{A}[\text{row}][\text{col}]$ is stored at index: $\text{index} = \text{row} \times N + \text{col}$
- This memory layout affects both CPU cache performance and GPU memory coalescing

3 Assignment

Part 1 – Coalesced memory access

- Measure execution time.
- Compute effective bandwidth for each kernel ($BW = \frac{\text{Total bytes transferred}}{\text{time}}$ in GB/s).
- Have a cpu implementation to verify the results and to compare the execution times.
- Repeat measurements to reduce noise and account for GPU warm-up effects.
- Observe how performance changes with matrix size and access pattern (which is faster and why?).

3 Assignment

Part 2 – Global, shared, and constant memory

- Design a kernel that multiplies two matrices.
- Assess the processing time for a kernel that utilizes global-only, global and shared, and global and constant memory.

Evaluations should consider changing the grid and/or input size.

3 Pointers – Part 2

Recall matrix multiplication:

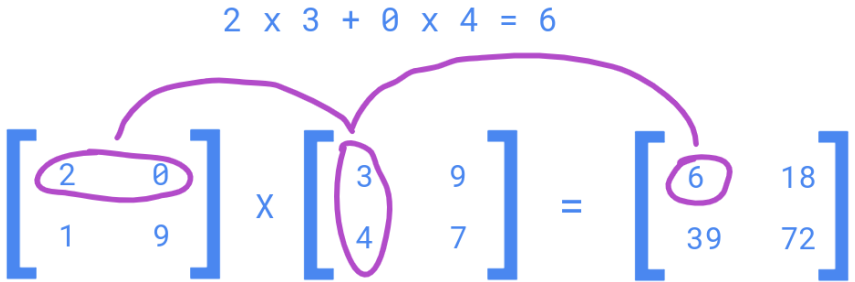


Image source: https://www.codingem.com/numpy-at-operator/

3 Pointers – Part 2

- Define an array that represents a matrix - in 1D or 2D form (start off with small square matrices and simple numbers for easier debugging).
- In this way define/allocate memory for three matrices, which will serve for $\mathbf{C} = \mathbf{AB}$.
- Make a kernel that uses only global memory to compute $\mathbf{C} \rightarrow$ **verify the results**.
- Make a second and third matrix multiplication kernel, which use shared and constant memory, respectively, in addition to global memory.
- Time the kernels for different input and/or grid sizes.

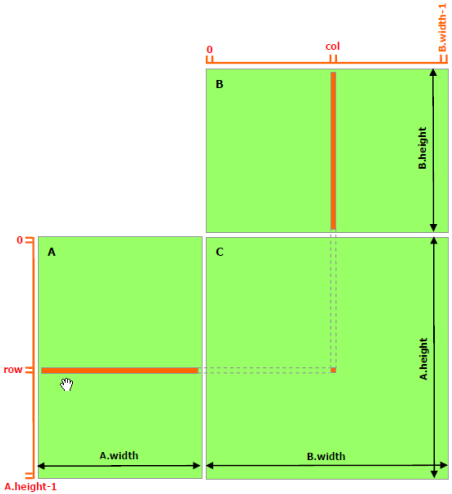
3 Guiding questions – Part 2

- How much time does memory migration take (CPU to GPU global; GPU global to GPU shared)?
- How do the execution times of matrix multiplication compare when employing the three different memory types?
- How do the times scale with input and/or grid size?

4 Outline

1. Intermezzo
2. GPU memory access and memory types
3. Exercise
4. Appendix
5. Sources

4 Memory access during matrix multiplication



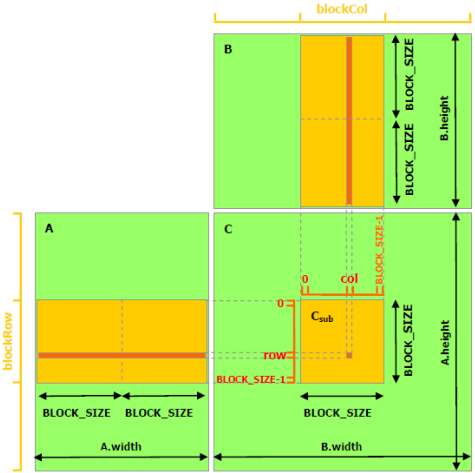
Assume:

- Matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ all NxN
- NxN grid of threads

Then:

- Each thread writes 1 element in $\mathbf{C}$
- $\mathbf{A}$-rows and $\mathbf{B}$-columns are re-read N-1 times

4 Improvement by leveraging shared memory



Assume:

- Matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ all NxN
- NxN grid of threads
- Square blocks with size BLOCK_SIZE

Then:

- Each thread writes 1 element in $\mathbf{C}$
- Each thread copies one element from both $\mathbf{A}$ and $\mathbf{B}$ to shared memory, k-times
- $\mathbf{A}$-rows and $\mathbf{B}$-columns are re-read k-times
- $k = \frac{N}{BLOCK_SIZE}$

5 Outline

1. Intermezzo
2. GPU memory access and memory types
3. Exercise
4. Appendix
5. Sources

5 Sources

- Coalesced meaning
- DRAM intro
- Coalesced access figure
- Blog post about unified access
- Blog post about shared memory
- GPU memory figure